

Group Project: Final submission

Team 02



Group Details

Full Name	BTH ID	Email
AHMED MD ASIF IQBAL	189979	mdah25@student.bth.se
Garcia Martinez Iker	191644	ikga25@student.bth.se
Runlin Long	190340	rulo25@student.bth.se
Letian Zhou	190341	lezh25@student.bth.se
Oghenero Jeremi Adjaino	191446	ogad25@student.bth.se

1. Project Description

1.1 Title

IKER Finance: Multi-Currency Personal Finance Manager.

1.2 Project Aim and Objectives

Project Aim:

To develop a personal finance application to help people who spend money abroad to better manage their financial expenses and budget. The goal is to promote improvement in users' spending habits while abroad for them to not end up spending more than they should.

Objectives:

- Track daily expenses and automatically convert between different currencies.
- To create budgets that work with multiple currencies.
- Categorize expenses to identify spending patterns to better manage their financial data.
- Generate and visualize reports and charts to understand spending habits over time.
- Build a secure system where users can safely store their financial data.

2. User Descriptions

Name	Description	Responsibilities
Multi-Currency User (Foreign students, international travelers, expatriate workers, immigrants and other individuals managing finances across multiple currencies)	Individuals who need to track expenses, manage budgets, and monitor financial activities while dealing with different currencies in their daily lives.	• Record daily transactions. • Set up and manage personal budgets. • Categorize expenses (food, transport, housing, etc.) • Monitor spending patterns across currencies. • Check financial reports and summaries. • Configure currency preferences.
System Admin	Responsible for maintaining the application infrastructure, settings, and providing user support.	• Manage user accounts and system access • Configure currency settings and exchange rates • Gather user feedback.

3. Software Features and Requirements

This application helps users track their money when they use different currencies. Users can add their daily expenses, and the app will show how much they spent in their home currency. (For example, if someone from Nigeria spends 100 SEK in Sweden, the app will automatically show them how much that is in Nigerian Naira). Users can organize their spending into different categories like food, transport, and housing to see where their money goes.

The app lets users create budgets for different time periods like weekly, monthly, quarterly, or yearly and sends warnings when they are close to spending over the budget limit. Users can see their spending history and budgets through a financial overview that shows their transaction summaries and budget status. The budget feature works with multiple currencies, so users can plan for both local expenses and money they want to send back home.

The application works on computers and phones and keeps all financial data safe and private. It also requires user authentication. The system admin can manage currency rates, review user feedback, and help users who have problems with the app. Everything is designed to be easy to use while keeping personal financial data secured.

3.1 Requirements

Multi-Currency User Requirements:

- FR1: Record and track transactions with amount, date, category, and optional descriptions
 - The app tracks transactions using the amount, a timestamp, category: bills, groceries, transport and allowing users to set an optional description
- FR2: View automatic conversion of all transactions to home currency
 - The app allows the user to compare their home currency with the currency of the country they are visiting
- FR3: Set budgets by category with multiple time periods
 - The app allows the user to set budgets for different categories such as groceries, transport, and bills. Users can choose time periods like weekly, monthly, quarterly, or yearly
- FR4: View financial overview with spending and budget summaries
 - The app contains a simple dashboard that allows users to easily access their expenses summaries and other options such as budgets
- FR5: Filter and search transactions by date range, category, or currency
 - The app allows users to easily search their transactions by providing time, category or even currency
- FR6: Update home currency and display preferences in settings

- The app allows users to easily update their home currency to any other currency they might need and to change their display settings
- FR7: Provide guidelines and FAQ for users
 - The app contains a Support section in the dashboard which contains guides to teach users how to use the system, moreover it also contains an FAQ section
- FR8: Collect user's reports and bug feedback
 - The app contains a section in the support section that allows users to submit error reports or bug feedback based on problems they may have encountered
- FR9: User authentication
 - The app requires users to create an account with a safe password to access the application, moreover the user logs in with token based authentication

System Administrator Requirements:

- FR12: Configure exchange rate sources and update frequently
 - The admin can manually update exchange rates, and each update contains a timestamp, these timestamps are shown when the rate is updated for them to frequently update and trace the exact time
- FR13: View and manage user feedback
 - The admins can view all the error reports and bug feedback that users provide in the support section, to better assist users and improve app functionality

3.2 Non-functional Requirements

Performance Requirements:

- NFR1: Page load time must be under 3 seconds
- NFR2: Transaction search/filter must return results within 1 second

Security Requirements:

- NFR3: API must use token based authentication (JWT)
- NFR4: Passwords must be encrypted
- NFR5: Transaction data transmission should be encrypted to protect user financial information

Usability Requirements:

- NFR6: All actions must provide user with feedback (success/error messages)

- NFR7: Currency amounts must display with proper symbols and formatting (example, \$1,234.56, 1.234,56 €)

Reliability Requirements:

- NFR8: Exchange rates must be manually updated at least once every 4 hours daily

Compatibility Requirements:

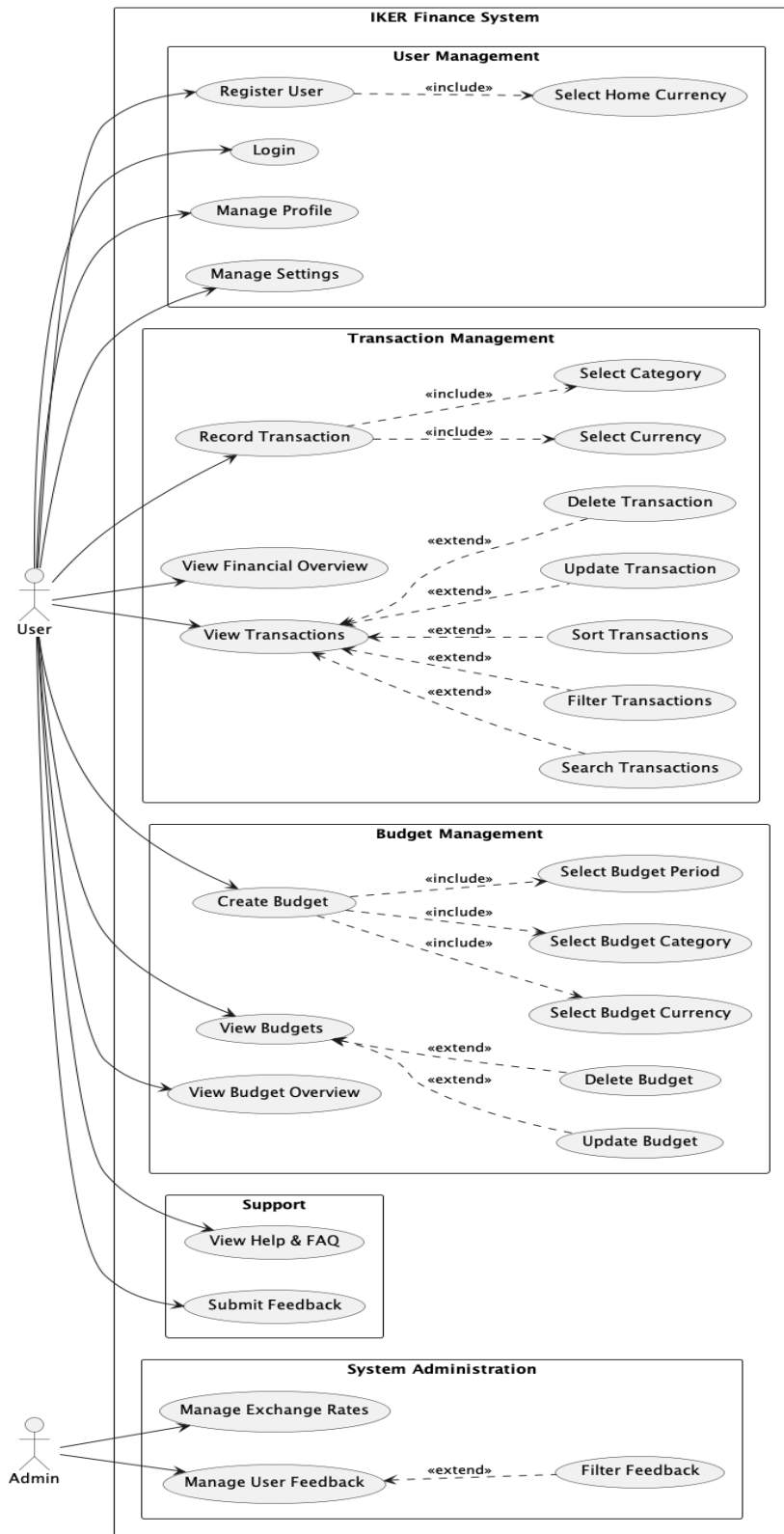
- NFR9: Must work on Chrome, Firefox, Safari, and Edge (latest 2 versions)
- NFR10: Must be accessible via web browser on desktops and mobile devices (no app installation required)

User Interface Requirements:

- NFR11: Application must be responsive and adapt to screen sizes from mobile to desktop
- NFR12: All user inputs must be validated before processing

IKER Finance - Use Case Analysis

1. UML Use Case Diagram



PlantUML Code:

```
@startuml
left to right direction
skinparam packageStyle rectangle

rectangle "IKER Finance System" {
    package "User Management" {
        usecase "Register User" as UC1
        usecase "Select Home Currency" as UC1a
        usecase "Login" as UC2
        usecase "Manage Profile" as UC3
        usecase "Manage Settings" as UC4
    }

    package "Transaction Management" {
        usecase "Record Transaction" as UC5
        usecase "Select Category" as UC5a
        usecase "Select Currency" as UC5b
        usecase "View Financial Overview" as UC6
        usecase "View Transactions" as UC7
        usecase "Search Transactions" as UC7a
        usecase "Filter Transactions" as UC7b
        usecase "Sort Transactions" as UC7c
        usecase "Update Transaction" as UC7d
        usecase "Delete Transaction" as UC7e
    }

    package "Budget Management" {
        usecase "Create Budget" as UC8
        usecase "Select Budget Category" as UC8a
        usecase "Select Budget Period" as UC8b
        usecase "Select Budget Currency" as UC8c
        usecase "View Budgets" as UC9
        usecase "Update Budget" as UC9a
        usecase "Delete Budget" as UC9b
        usecase "View Budget Overview" as UC10
    }

    package "Support" {
        usecase "View Help & FAQ" as UC11
        usecase "Submit Feedback" as UC12
    }
}
```



```
package "System Administration" {  
    usecase "Manage Exchange Rates" as UC13  
    usecase "Manage User Feedback" as UC14  
    usecase "Filter Feedback" as UC14a  
}  
}
```

```
actor "User" as User  
actor "Admin" as Admin
```

```
User --> UC1  
User --> UC2  
User --> UC3  
User --> UC4  
User --> UC5  
User --> UC6  
User --> UC7  
User --> UC8  
User --> UC9  
User --> UC10  
User --> UC11  
User --> UC12
```

```
Admin --> UC13  
Admin --> UC14
```

```
UC1 ..> UC1a : <<include>>  
UC5 ..> UC5a : <<include>>  
UC5 ..> UC5b : <<include>>  
UC8 ..> UC8a : <<include>>  
UC8 ..> UC8b : <<include>>  
UC8 ..> UC8c : <<include>>
```

```
UC7 <.. UC7a : <<extend>>  
UC7 <.. UC7b : <<extend>>  
UC7 <.. UC7c : <<extend>>  
UC7 <.. UC7d : <<extend>>  
UC7 <.. UC7e : <<extend>>  
UC9 <.. UC9a : <<extend>>  
UC9 <.. UC9b : <<extend>>  
UC14 <.. UC14a : <<extend>>
```

```
@enduml
```

2. High-Level Use Cases

For Multi-Currency User

UC1:

Use case name	Register User
Use case ID:	UC1
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User creates a new account by providing email, password, first name, last name, and selecting their home currency. The home currency is mandatory and will be used for all budget calculations and reporting. Once selected during registration, the home currency cannot be changed. System validates the information, creates the account, and grants access to the application.

UC2:

Use case name	Login
Use case ID:	UC2
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User enters their email and password to log into the system. System verifies the credentials and grants access to the application.

UC3:

Use case name	Manage Profile
Use case ID:	UC3
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User views and updates their profile information including first name and last name. Multi-Currency User can also change their password by providing current password and new password. System validates and saves the changes. The home currency is displayed but cannot be modified.

UC4:

Use case name	Manage Settings
Use case ID:	UC4
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User views and updates their application preferences including default transaction currency and time zone. System saves and applies the new settings. The home currency is displayed for reference but cannot be changed.

UC5:

Use case name	Record Transaction
Use case ID:	UC5
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User enters transaction details including amount, currency, date, category, and optional notes. Multi-Currency User must select a category from the predefined list (Food, Transport, Shopping, Bills, Entertainment, and others) and must select a currency. System validates the information, saves the transaction with both original currency and home currency amounts, and updates spending totals and budget tracking.

UC6:

Use case name	View Financial Overview
Use case ID:	UC6
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User accesses the main overview page to see a comprehensive financial snapshot. System displays transaction summaries (total spent, comparison with last month), budget summaries (budget status, budgets at risk), recent transactions preview, and active budgets list. All amounts are displayed in the home currency.

UC7:

Use case name	View Transactions
Use case ID:	UC7
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User accesses the complete transaction list page showing all past transactions with their details (description, amount in original currency, converted amount, category, date). System displays the list with pagination. Multi-Currency User can optionally search transactions by keywords, filter by category/currency/date range, sort by different fields, update existing transactions, or delete transactions from this view.

UC8:

Use case name	Create Budget
Use case ID:	UC8
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User creates a new budget by selecting one category, choosing a time period (Weekly, Monthly, Quarterly, or Yearly), selecting a currency, entering a budget amount, and setting start date. Each budget applies to a single category for the specified period. System saves the budget and automatically starts tracking spending against it with default alert thresholds (80% warning, 100% critical). System displays average spending for the selected category from previous periods to help user decide the budget amount.

UC9:

Use case name	View Budgets
Use case ID:	UC9
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User accesses the budget management page to view all created budgets. System displays a list showing category, amount, spent, remaining, progress percentage, period type, and duration for each budget. Multi-Currency User can optionally update or delete budgets from this view. System shows budgets with pagination.

UC10:

Use case name	View Budget Overview
Use case ID:	UC10
Actor(s)	Multi-Currency User
Type:	Primary
Basic Flow:	Multi-Currency User views detailed budget analysis and spending metrics. System displays budget summary showing total budgeted amount, total spent, remaining amount, percentage used, and alert status. Budget alerts (80% warning and 100% critical) are automatically displayed based on spending progress. All amounts are shown in the home currency.

UC11:

Use case name	View Help & FAQ
Use case ID:	UC11
Actor(s)	Multi-Currency User
Type:	Secondary
Basic Flow:	Multi-Currency User accesses the help section to read instructions and frequently asked questions. System displays guidance on using the application features, including transaction recording, budget management, and currency handling.

UC12:

Use case name	Submit Feedback
Use case ID:	UC12
Actor(s)	Multi-Currency User
Type:	Secondary
Basic Flow:	Multi-Currency User provides feedback about the application by selecting feedback type (Bug, Feature, Improvement, Question, Complaint), setting priority (Low, Medium, High, Critical), entering subject and detailed description. System validates and saves the submission for admin review.

For System Admin

UC13:

Use case name	Manage Exchange Rates
Use case ID:	UC13
Actor(s)	System admin
Type:	Primary
Basic Flow:	System Admin manages currency exchange rates by viewing all exchange rates, creating new exchange rate entries, updating existing rates, or deleting obsolete rates. When creating or updating, admin specifies source currency, target currency, rate value, and effective date. System validates the information, saves the exchange rate, and makes it available for transaction conversions. All exchange rate changes are tracked with timestamp and admin user ID for audit purposes.

UC14:

Use case name	Manage User Feedback
Use case ID:	UC14
Actor(s)	System admin
Type:	Secondary
Basic Flow:	System Admin reviews and manages all user feedback submissions. System displays all feedback with user details, type, priority, status, and submission date. System Admin can optionally filter feedback by type, status, priority, or date range. System Admin can update feedback status (Open, In Progress, Responded, Resolved, Closed) and provide a response to the user. System saves the status update, admin response, response date, and responding admin ID.

3. Expanded Use Cases

Expanded Use Case 5: Record Transaction

Use Case Name	Record Transaction
Use Case ID	UC5
Actor(s)	Multi-Currency User
Purpose	To save a money transaction with category and currency selection
Overview	A Multi-Currency User enters details about money they spent or received. The system validates the information and saves it for tracking and budget calculations.
Type	Primary
Cross Reference	FR1, FR2, NFR6, NFR7, NFR13
Pre-Conditions	• Multi-Currency User is logged into the system • Multi-Currency User has set their home currency during registration • System has current exchange rates available • Categories are defined in the system
Main Success Scenario	
Actor's Actions	System's Response
1) Multi-Currency User clicks "Add Transaction" button on the main page	2) System shows a form with fields for amount, currency dropdown list, date picker, category dropdown list, and optional notes/description text box
3) Multi-Currency User enters the transaction amount (for example, 100 SEK)	4) System checks that the amount is a number and is more than zero
5) Multi-Currency User selects the currency from the dropdown list	6) System displays the amount and stores both the original currency and home currency for the transaction
7) Multi-Currency User selects the transaction date using the date picker	8) System checks the date is not in the future
9) Multi-Currency User selects a category from the predefined list (such as Food, Transport, Shopping, Bills, Entertainment, and others)	10) System confirms the category selection

11) Multi-Currency User optionally adds notes/description and clicks "Save" button	12) System validates all required fields, saves the transaction with both original and converted amounts, updates total spending, updates budget tracking if active budgets exist for the selected category, and shows message: "Transaction saved successfully"
13) Multi-Currency User sees the success message and returns to main page	14) System updates the main page to show the new transaction in the list and updates spending totals
Post-Condition(s)	• Transaction is saved in the database with both currency amounts (original and home currency) • Spending totals are updated • Budget status is recalculated if active budgets exist for the selected category • Transaction appears in the transaction list • If budget alerts are triggered (80% or 100%), they are displayed on budget overview
Extensions (if any)	
Actor's Actions	System's Response
3a) Multi-Currency User enters letters or special characters instead of numbers	3a.1) System shows error: "Please enter a valid number" and marks the amount field in red
5a) System cannot find exchange rate for chosen currency pair	5a.1) System shows error: "Exchange rate not available for this currency pair. Please contact administrator or try a different currency"
7a) Multi-Currency User selects a future date	7a.1) System shows warning: "Cannot use future dates" and resets date to current date
9a) Multi-Currency User does not select a category	9a.1) System shows error: "Category is required" and prevents saving
11a) Multi-Currency User clicks Save without filling required fields	11a.1) System highlights all missing required fields in red and shows message: "Please fill all required fields"

Expanded Use Case 8: Create Budget

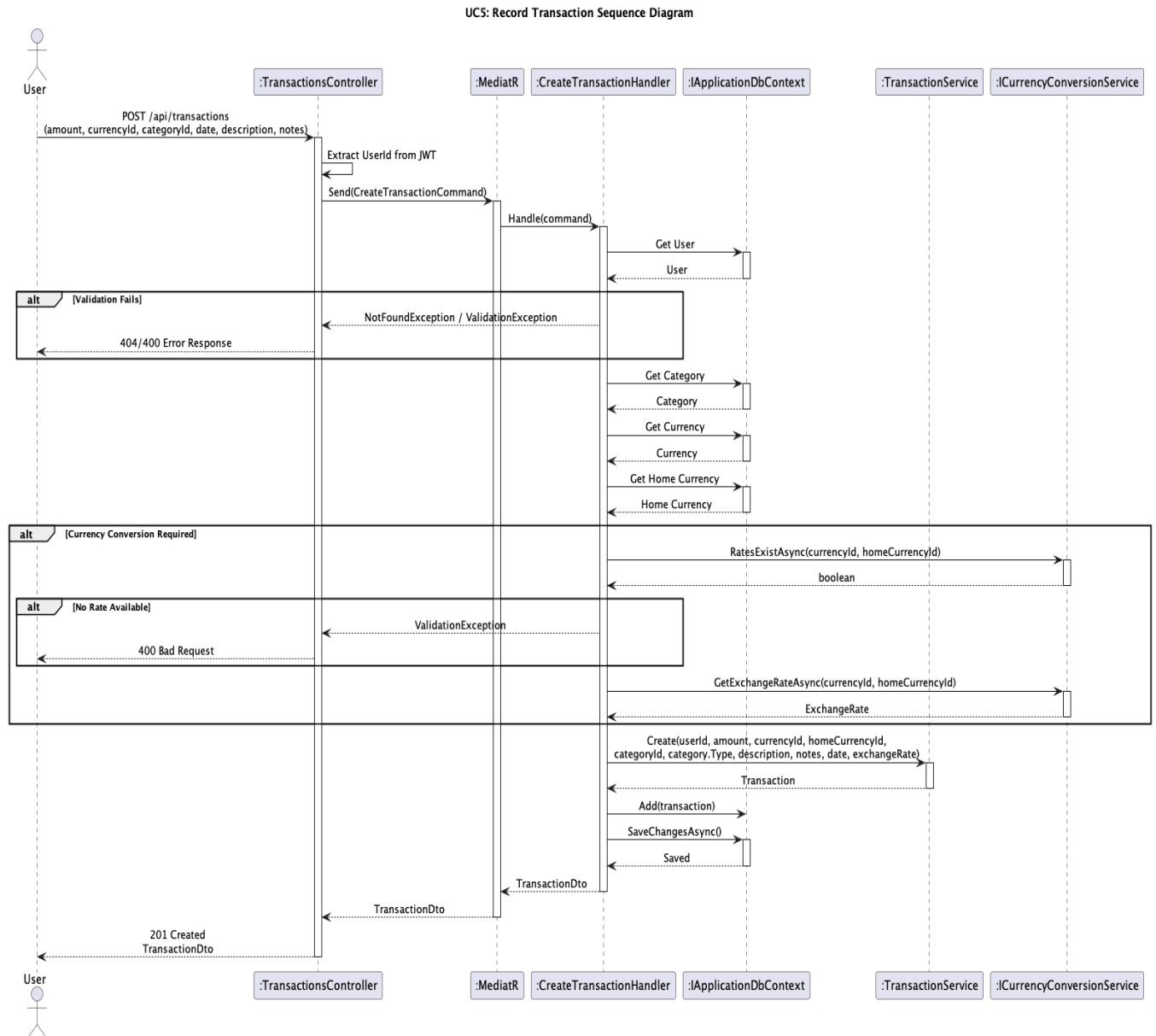
Use Case Name	Create Budget
Use Case ID	UC8
Actor(s)	Multi-Currency User
Purpose	To set spending limits for a specific category and time period for better money management
Overview	A Multi-Currency User creates a budget for a specific category and time period. The system saves the budget settings and automatically tracks actual spending against the limit with automatic alert thresholds.
Type	Primary
Cross Reference	FR3, NFR1, NFR6, NFR13
Pre-Conditions	• Multi-Currency User is logged into the system • Multi-Currency User has set up their profile with home currency • Categories are defined in the system
Main Success Scenario	
Actor's Actions	System's Response
1) Multi-Currency User navigates to "Budget Management" from the main menu	2) System shows the budget page with existing budgets (if any) and a "Create New Budget" button
3) Multi-Currency User clicks "Create New Budget"	4) System shows form with fields for category dropdown, time period options (Weekly, Monthly, Quarterly, Yearly), budget currency dropdown, budget amount field, and start date picker
5) Multi-Currency User selects a time period (for example, Monthly)	6) System confirms the period selection
7) Multi-Currency User selects one category from the dropdown list (for example, "Food")	8) System shows the average spending for the selected category from previous periods to help user decide the budget amount
9) Multi-Currency User selects budget currency from the dropdown	10) System confirms currency selection
11) Multi-Currency User enters budget amount	12) System checks that the amount is more than zero

13) Multi-Currency User selects start date	14) System checks the date and calculates the end date based on selected period
15) Multi-Currency User clicks "Create Budget"	16) System creates the budget, calculates current period spending for the selected category, automatically sets alert thresholds (80% warning, 100% critical), saves all information, and shows message: "Budget created successfully"
17) Multi-Currency User views the budget summary	18) System shows the new budget with category, amount, current spending, remaining amount, progress bar, period, and duration
Post-Condition(s)	• Budget is created and active for the chosen period • System starts tracking spending against the budget for the selected category • Alert thresholds are automatically set at 80% (warning) and 100% (critical) • Budget appears on budget list • Current period spending is automatically calculated and tracked against the new budget • If current spending exceeds alert thresholds, alerts are displayed on budget overview
Extensions (if any)	
Actor's Actions	System's Response
11a) Multi-Currency User enters zero or negative number	11a.1) System shows error: "Budget amount must be more than zero" and marks the amount field in red
11b) Multi-Currency User enters a very large amount (for example, weekly budget over 1,000,000 in home currency)	11b.1) System shows warning: "This amount seems unusually high. Are you sure?" with "Confirm" and "Change" buttons
13a) Multi-Currency User tries to set start date in the past (more than current period)	13a.1) System shows error: "Start date cannot be in the past. Budgets can only be set for current or future periods" and resets to current date
15a) Multi-Currency User tries to create a second budget for the same category and overlapping period	15a.1) System shows error: "You already have a budget for this category in this period. Do you want to update the existing budget?" with "Update" and "Cancel" buttons
15b) Multi-Currency User clicks Create without filling required fields	15b.1) System highlights all missing required fields and shows message: "Please fill all required fields (category, period, currency, amount)"

IKER Finance - Design documentation

UML Interaction Diagrams

UC5: Record Transaction Sequence Diagram



PlantUML Code

```
@startuml

skinparam sequenceMessageAlign center

title UC5: Record Transaction Sequence Diagram

actor User
participant ":TransactionsController" as Controller
participant ":MediatR" as Mediator
participant ":CreateTransactionHandler" as Handler
participant ":IApplicationDbContext" as Context
participant ":TransactionService" as TxService
participant ":ICurrencyConversionService" as ConvService

User -> Controller: POST /api/transactions\n(amount, currencyId, categoryId, date, description, notes)
activate Controller

Controller -> Controller: Extract UserId from JWT
Controller -> Mediator: Send(CreateTransactionCommand)
activate Mediator

Mediator -> Handler: Handle(command)
activate Handler

Handler -> Context: Get User
activate Context
Context --> Handler: User
deactivate Context

alt Validation Fails
    Handler --> Controller: NotFoundException / ValidationException
    Controller --> User: 404/400 Error Response
end

    Handler -> Context: Get Category
    activate Context
    Context --> Handler: Category
    deactivate Context

    Handler -> Context: Get Currency
    activate Context
    Context --> Handler: Currency
    deactivate Context
```

```

    Handler --> Context: Get Home Currency
    activate Context
    Context --> Handler: Home Currency
    deactivate Context

alt Currency Conversion Required
    Handler --> ConvService: RatesExistAsync(currencyId, homeCurrencyId)
    activate ConvService
    ConvService --> Handler: boolean
    deactivate ConvService

alt No Rate Available
    Handler --> Controller: ValidationException
    Controller --> User: 400 Bad Request
    end

    Handler --> ConvService: GetExchangeRateAsync(currencyId, homeCurrencyId)
    activate ConvService
    ConvService --> Handler: ExchangeRate
    deactivate ConvService
    end

    Handler --> TxService: Create(userId, amount, currencyId, homeCurrencyId, \ncategoryId, category.Type,
description, notes, date, exchangeRate)
    activate TxService
    TxService --> Handler: Transaction
    deactivate TxService

    Handler --> Context: Add(transaction)
    Handler --> Context: SaveChangesAsync()
    activate Context
    Context --> Handler: Saved
    deactivate Context

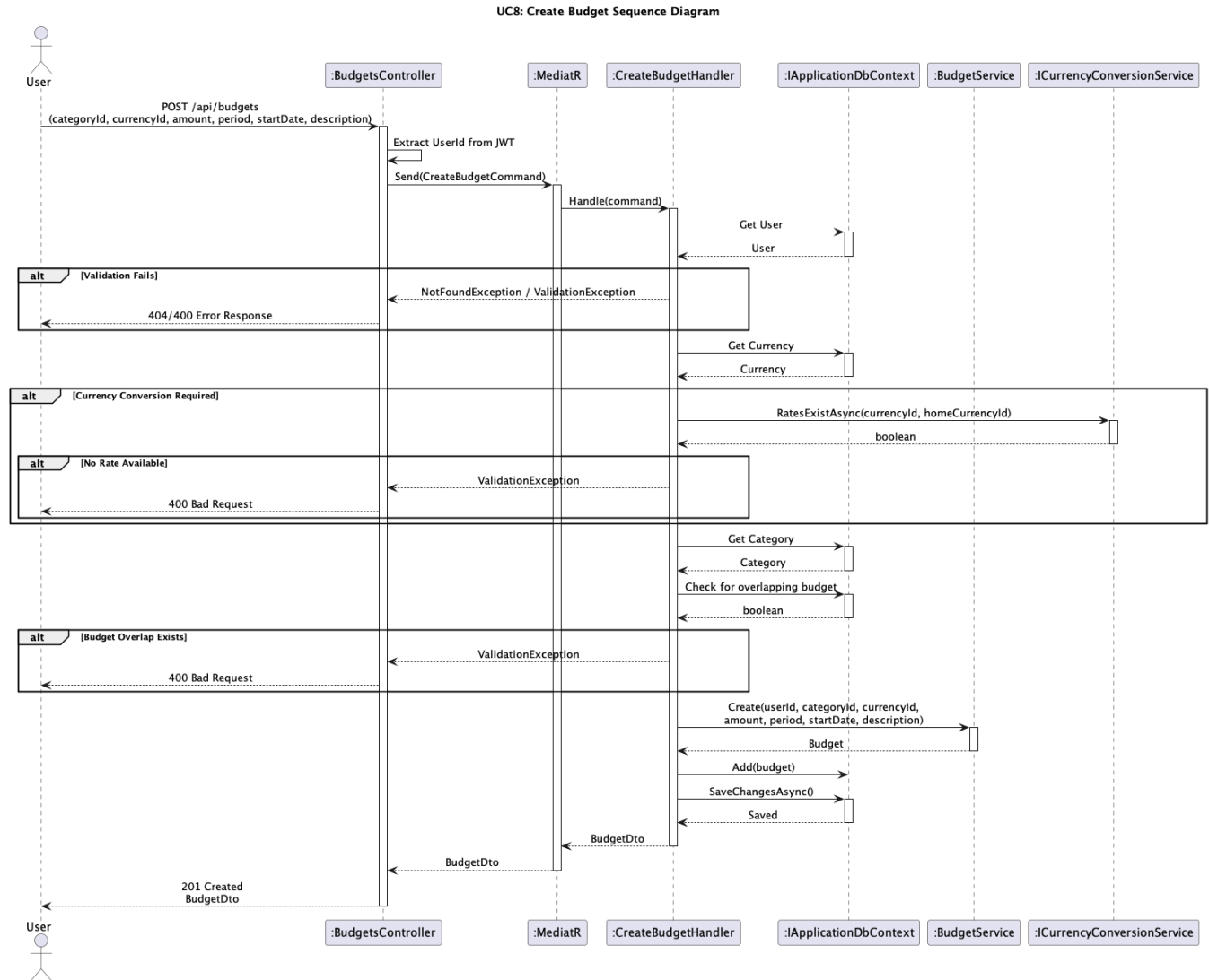
    Handler --> Mediator: TransactionDto
    deactivate Handler

    Mediator --> Controller: TransactionDto
    deactivate Mediator

    Controller --> User: 201 Created\nTransactionDto
    deactivate Controller

```


UC8: Create Budget Sequence Diagram



PlantUML Code

```
@startuml

skinparam sequenceMessageAlign center

title UC8: Create Budget Sequence Diagram

actor User
participant ":BudgetsController" as Controller
participant ":MediatR" as Mediator
participant ":CreateBudgetHandler" as Handler
participant ":IApplicationDbContext" as Context
participant ":BudgetService" as BudgetService
participant ":ICurrencyConversionService" as ConvService

User -> Controller: POST /api/budgets\n(categoryId, currencyId, amount, period, startDate, description)
activate Controller

Controller -> Controller: Extract UserId from JWT
Controller -> Mediator: Send(CreateBudgetCommand)
activate Mediator

Mediator -> Handler: Handle(command)
activate Handler

Handler -> Context: Get User
activate Context
Context --> Handler: User
deactivate Context

alt Validation Fails
    Handler --> Controller: NotFoundException / ValidationException
    Controller --> User: 404/400 Error Response
end

    Handler -> Context: Get Currency
    activate Context
    Context --> Handler: Currency
    deactivate Context

alt Currency Conversion Required
    Handler -> ConvService: RatesExistAsync(currencyId, homeCurrencyId)
    activate ConvService
    ConvService --> Handler: boolean
```

```

    deactivate ConvService

alt No Rate Available
    Handler --> Controller: ValidationException
    Controller --> User: 400 Bad Request
    end
end

    Handler -> Context: Get Category
    activate Context
    Context --> Handler: Category
    deactivate Context

    Handler -> Context: Check for overlapping budget
    activate Context
    Context --> Handler: boolean
    deactivate Context

alt Budget Overlap Exists
    Handler --> Controller: ValidationException
    Controller --> User: 400 Bad Request
    end

    Handler -> BudgetService: Create(userId, categoryId, currencyId,\namount, period, startDate, description)
    activate BudgetService
    BudgetService --> Handler: Budget
    deactivate BudgetService

    Handler -> Context: Add(budget)
    Handler -> Context: SaveChangesAsync()
    activate Context
    Context --> Handler: Saved
    deactivate Context

    Handler --> Mediator: BudgetDto
    deactivate Handler

    Mediator --> Controller: BudgetDto
    deactivate Mediator

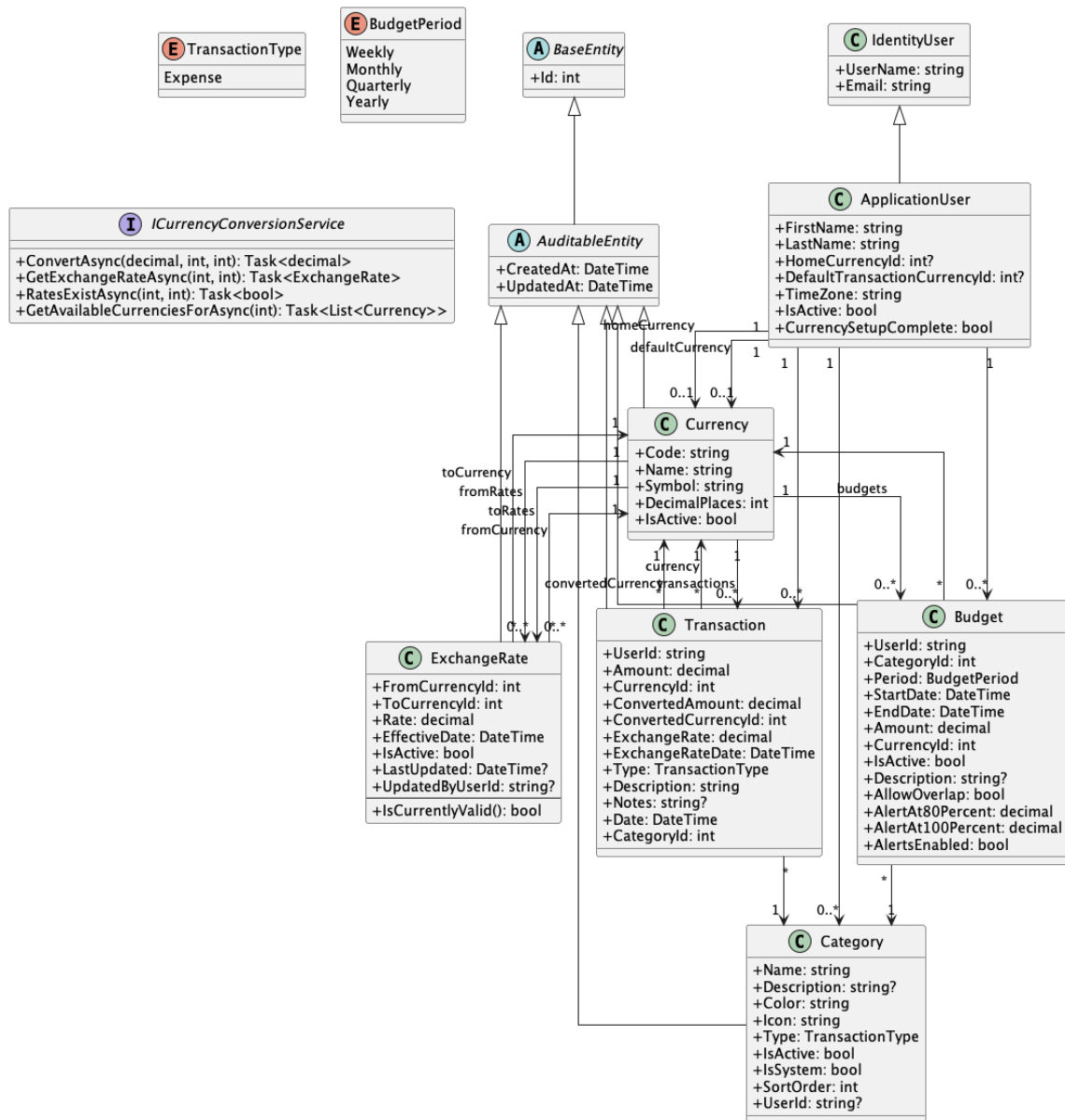
    Controller --> User: 201 Created\nBudgetDto
    deactivate Controller

@enduml

```

Class Diagram for UC5 (Record Transaction) and UC8 (Create Budget)

Class Diagram for UC5 (Record Transaction) and UC8 (Create Budget)



PlantUML Code

```
@startuml

skinparam classAttributeIconSize 0
skinparam linetype ortho

title Class Diagram for UC5 (Record Transaction) and UC8 (Create Budget)

abstract class BaseEntity {
    + Id: int
}

abstract class AuditableEntity {
    + CreatedAt: DateTime
    + UpdatedAt: DateTime
}

class IdentityUser {
    + UserName: string
    + Email: string
}

class ApplicationUser {
    + FirstName: string
    + LastName: string
    + HomeCurrencyId: int?
    + DefaultTransactionCurrencyId: int?
    + TimeZone: string
    + IsActive: bool
    + CurrencySetupComplete: bool
}

class Currency {
    + Code: string
    + Name: string
    + Symbol: string
    + DecimalPlaces: int
    + IsActive: bool
}

class ExchangeRate {
    + FromCurrencyId: int
    + ToCurrencyId: int
    + Rate: decimal
    + EffectiveDate: DateTime
}
```

```

+ IsActive: bool
+ LastUpdated: DateTime?
+ UpdatedByUserId: string?
--
+ IsCurrentlyValid(): bool
}

class Transaction {
+ UserId: string
+ Amount: decimal
+ CurrencyId: int
+ ConvertedAmount: decimal
+ ConvertedCurrencyId: int
+ ExchangeRate: decimal
+ ExchangeRateDate: DateTime
+ Type: TransactionType
+ Description: string
+ Notes: string?
+ Date: DateTime
+ CategoryId: int
}

class Category {
+ Name: string
+ Description: string?
+ Color: string
+ Icon: string
+ Type: TransactionType
+ IsActive: bool
+ IsSystem: bool
+ SortOrder: int
+ UserId: string?
}

class Budget {
+ UserId: string
+ CategoryId: int
+ Period: BudgetPeriod
+ StartDate: DateTime
+ EndDate: DateTime
+ Amount: decimal
+ CurrencyId: int
+ IsActive: bool
+ Description: string?
+ AllowOverlap: bool
+ AlertAt80Percent: decimal

```

```

        + AlertAt100Percent: decimal
        + AlertsEnabled: bool
    }

enum TransactionType {
    Expense
}

enum BudgetPeriod {
    Weekly
    Monthly
    Quarterly
    Yearly
}

interface ICurrencyConversionService {
    + ConvertAsync(decimal, int, int): Task<decimal>
    + GetExchangeRateAsync(int, int): Task<ExchangeRate>
    + RatesExistAsync(int, int): Task<bool>
    + GetAvailableCurrenciesForAsync(int): Task<List<Currency>>
}

BaseEntity <|-- AuditableEntity
AuditableEntity <|-- Currency
AuditableEntity <|-- ExchangeRate
AuditableEntity <|-- Transaction
AuditableEntity <|-- Category
AuditableEntity <|-- Budget

IdentityUser <|-- ApplicationUser

ApplicationUser "1" --> "0..1" Currency : homeCurrency
ApplicationUser "1" --> "0..1" Currency : defaultCurrency
ApplicationUser "1" --> "0..*" Transaction
ApplicationUser "1" --> "0..*" Category
ApplicationUser "1" --> "0..*" Budget

Currency "1" --> "0..*" ExchangeRate : fromRates
Currency "1" --> "0..*" ExchangeRate : toRates
Currency "1" --> "0..*" Transaction : transactions
Currency "1" --> "0..*" Budget : budgets

Transaction "*" --> "1" Category
Transaction "*" --> "1" Currency : currency
Transaction "*" --> "1" Currency : convertedCurrency

```

Budget $\frac{11}{11}$ --> "1" Category

Budget $\frac{11}{11}$ --> "1" Currency

ExchangeRate $\frac{11}{11}$ --> "1" Currency : fromCurrency

ExchangeRate $\frac{11}{11}$ --> "1" Currency : toCurrency

@enduml

IKER Finance - Quality Assurance Activity Report, including unit tests

Scope of Testing

This report provides comprehensive testing coverage across **the entire application**, mainly focusing on our primary use cases: **UC5 (Record Transaction)** and **UC8 (Create Budget)**.

Our testing strategy focused on all major features of IKER Finance including:

- **Transaction Management (UC5)** - Core financial tracking with multi-currency support
- **Budget Management (UC8)** - Budget creation, tracking, and alerts
- **User Management (UC1, UC3, UC4)** - Registration, profile, and settings
- **Authentication (UC2)** - Secure login and access control
- **Exchange Rate Management (UC13)** - Admin functionality for currency conversion rates
- **Feedback System (UC12, UC14)** - User feedback collection and management
- **Financial Overview (UC6, UC7, UC9, UC10)** - Dashboard, transaction viewing, and budget analytics

With **326 unit tests**, **42 integration tests**, and **9 architecture tests**, totaling **377 passing tests**, we have achieved comprehensive coverage across all application layers.

Question 1: Unit Tests for Project Code Units

Total Unit Tests Written

We have written a total of **326 unit tests** covering **all implemented features** across the entire application. Our testing strategy covers all code units in all the use cases, including business logic, data processing, validation components, domain services, and API controllers.

Test Coverage Summary

Metric	UC5: Record Transaction	UC8: Create Budget	UC13: Manage Exchange Rates	UC12/UC14 : Feedback	UC1/UC3 /UC4: Users	UC2: Auth	Others	Total
Unit Tests	62 tests	91 tests	37 tests	36 tests	28 tests	20 tests	52 tests	326 tests
Integration Tests	9 tests	12 tests	1 test	0 tests	10 tests	5 tests	5 tests	42 tests

Coverage Tool Used: Coverlet with XPlat Code Coverage collector and xUnit testing framework

Additional Tests:

- **Architecture Tests:** 9 tests ensuring architectural constraints and clean architecture principles

Use Case 5: Record Transaction (62 Unit Tests)

This use case allows users to record financial transactions with automatic currency conversion. The 62 unit tests cover:

Transaction Operations (25 tests):

- Creating new transactions: 10 tests
- Updating existing transactions: 10 tests
- Deleting transactions: 5 tests

Data Retrieval (21 tests):

- Getting transaction lists: 8 tests
- Getting individual transactions: 6 tests
- Getting transaction summaries: 7 tests

Input Validation (16 tests):

- Amount validation (positive values, decimal precision)
- Date validation (not future dates for expenses)
- Required fields validation
- Description length checks
- Currency existence validation
- Category existence validation

Use Case 8: Create Budget (91 Unit Tests)

This use case allows users to create and manage budgets with different time periods (Weekly, Monthly, Yearly) and track spending against budget limits. The 91 unit tests cover:

Budget Operations (28 tests):

- Creating new budgets: 14 tests

- Updating existing budgets: 9 tests
- Deleting budgets: 5 tests

Data Retrieval (32 tests):

- Getting budget lists: 8 tests
- Getting individual budgets: 5 tests
- Getting active budgets: 6 tests
- Getting budget summaries: 6 tests
- Previewing budget impacts: 7 tests

Input Validation (26 tests):

- Budget creation validation: 14 tests
- Budget update validation: 12 tests

Business Logic Tests:

- Budget period calculations (Weekly/Monthly/Yearly)
- Alert threshold triggering (80%, 100%)
- Budget-to-spending comparisons
- Currency conversion in budget tracking

Exchange Rate Management (37 Unit Tests)

Admin functionality for managing currency exchange rates (UC13):

Operations (9 tests):

- Creating exchange rates: 5 tests
- Deleting exchange rates: 2 tests
- Querying exchange rates: 2 tests

Input Validation (28 tests):

- Create validation (UC13): 10 tests
- Update validation (UC13): 8 tests
- Delete validation (UC13): 7 tests

Feedback System (36 Unit Tests)

User feedback collection and management system (UC12, UC14):

Operations (11 tests):

- Creating feedback (UC12): 4 tests
- Updating feedback status (UC14): 4 tests
- Querying feedbacks (UC14): 3 tests

Input Validation (25 tests):

- Create feedback validation (UC12): 13 tests
- Update status validation (UC14): 12 tests

User Profile Management (28 Unit Tests)

User account settings and profile management (UC3, UC4):

Input Validation:

- Change password validation (UC3): 8 tests
- Update user settings validation (UC4): 10 tests
- Update user profile validation (UC3): 10 tests

Authentication (20 Unit Tests)

User registration and login functionality (UC1, UC2):

Operations:

- Registration (UC1): 10 tests
- Login (UC2): 10 tests

Other Features (52 Unit Tests)

Additional tests covering supporting use cases (UC6, UC7, UC9, UC10):

Query and Display Operations:

- Financial overview queries (UC6): Tests for dashboard summaries
- Transaction viewing and filtering (UC7): Tests for search, filter, sort operations
- Budget listing and display (UC9): Tests for budget view and management
- Budget overview analytics (UC10): Tests for budget analysis and metrics
- Category and currency management: Tests for reference data access

Sample Unit Test 1: Transaction Currency Conversion (UC5)

This test checks the core currency conversion feature from UC5 and makes sure transactions in foreign currencies are correctly converted to the user's home currency:

```
[Fact]
public async Task Handle_WithDifferentCurrency_AppliesExchangeRate()
{
    var command = new CreateTransactionCommand
    {
        UserId = "user123",
```

```

        Amount = 100m,
        CurrencyId = 2,
        CategoryId = 1,
        Date = DateTime.UtcNow,
        Description = "Test"
    };

    var user = new ApplicationUser { Id = "user123", HomeCurrencyId = 1 };
    var category = new Category { Id = 1, Name = "Food", Type = TransactionType.Expense };
    var transactionCurrency = new Currency { Id = 2, Code = "SEK", Symbol = "kr", IsActive = true };
    var homeCurrency = new Currency { Id = 1, Code = "BDT", Symbol = "৳", IsActive = true };
    var exchangeRate = new ExchangeRate { FromCurrencyId = 2, ToCurrencyId = 1, Rate = 10.5m, IsActive =
true };

    SetupMockDbSets(new[] { user }, new[] { category }, new[] { transactionCurrency, homeCurrency }, new[]
{ exchangeRate });

    _mockConversionService.Setup(x => x.RatesExistAsync(2, 1)).ReturnsAsync(true);
    _mockConversionService.Setup(x => x.GetExchangeRateAsync(2, 1)).ReturnsAsync(exchangeRate);

    var result = await _handler.Handle(command, CancellationToken.None);

    result.Amount.Should().Be(100m);
    result.ConvertedAmount.Should().Be(1050m);
    result.ExchangeRate.Should().Be(10.5m);
    result.CurrencyCode.Should().Be("SEK");
    result.ConvertedCurrencyCode.Should().Be("BDT");
}

```

Sample Unit Test 2: Budget Amount Validation (UC8)

This test verifies budget amount input and ensures the system prevents users from creating budgets with zero or negative values:

```
[Fact]
public async Task Validate_WithZeroAmount_ShouldFail()
{
    var command = new CreateBudgetCommand
    {
        UserId = "user123",
        CategoryId = 1,
        CurrencyId = 1,
        Amount = 0m,
        Period = BudgetPeriod.Monthly,
        StartDate = DateTime.UtcNow
    };

    var result = await _validator.ValidateAsync(command);

    result.IsValid.Should().BeFalse();
    result.Errors.Should().ContainSingle(e => e.PropertyName == "Amount")
        .Which.ErrorMessage.Should().Be("Budget amount must be greater than zero");
}
```

Question 2: Code Coverage Percentage

Overall Project Coverage

The overall project code coverage statistics generated by Coverlet are as follows:

Layer	Line Coverage	Branch Coverage	Method Coverage
Overall Project	26.6%	57.0%	88.6%
IkerFinance.API	75.6%	-	-
IkerFinance.Application	94.4%	-	-
IkerFinance.Domain	63.8%	-	-
IkerFinance.Infrastructure	5.6%	-	-

Question 3: Integration Tests

Integration Tests Written

Yes, we have written **42 integration tests** covering all major features across the application:

Feature	Integration Tests	Coverage
Budgets (UC8)	12 tests	Primary use case - extensively tested
Users (UC1, UC3, UC4)	10 tests	Profile, settings, password management
Transactions (UC5)	9 tests	Primary use case - extensively tested
Auth (UC2)	5 tests	Registration, login, token validation
Categories	3 tests	Category listing and retrieval
Currencies	2 tests	Currency data access
Exchange Rates (UC13)	1 test	Basic rate management
Total	42 tests	

What These Integration Tests Check

Our integration tests verify the complete **end-to-end functionality** of the application:

1. Full Flow from HTTP Request to Database:

- Tests use a real HttpClient to send requests to API endpoints
- Requests traverse the entire application stack: API → Application → Domain → Infrastructure → Database
- Tests check data is correctly saved in the database and can be retrieved

2. Authentication and Authorization:

- Tests verify endpoints require proper authentication tokens
- They use Authentication helpers register users and obtain JWT access tokens
- Tests ensure unauthorized requests return 401 Unauthorized
- Role-based authorization (Admin vs. User) is validated

3. Multiple Components Working Together:

- **Currency conversion integration (UC5):** API → CurrencyConversionService → ExchangeRateRepository → Database
- **Budget tracking with transactions (UC5 + UC8):** Transaction creation updates budget spending calculations

- **User profile management:** Profile updates → Database → Retrieval validation
- Database operations, user authentication, transaction creation, and budget calculations all work together

4. Validation Error Handling:

- Tests verify proper validation error responses (400 Bad Request)
- Error messages are correctly formatted and returned to client
- FluentValidation integration is working correctly

5. Data Consistency:

- Tests verify referential integrity (foreign keys work correctly)
- Cascade operations function correctly

Question 4: Acceptance Tests

Yes, we conduct regular acceptance testing for our use cases to ensure they meet the requirements. However, we have not formally documented these acceptance tests yet. During our acceptance testing activities, we found one issue related to input validation for large amounts, which has been fixed and is documented in Question 5.

Question 5: Bug Statistics

Bugs Found During Testing

During our testing activities, we found a total of 2 bugs: one during unit testing and one during acceptance testing. Both bugs have been fixed and verified. (Some issues were encountered during development, but those were fixed before completing the tasks and before these formal testing activities began.)

Bug 1: Precision Loss in Budget Calculations (Found in Unit Testing)

The Problem:

When calculating budget summaries and previewing budget impacts, the system was performing unnecessary currency conversions even when the transaction currency matched the budget currency. This caused precision loss because the system was converting transaction amounts through multiple steps:

1. First conversion: transaction amount to home currency

2. Second conversion: home currency to budget currency

This double conversion was not needed when the transaction and budget used the same currency.

For example, if a user made a transaction of 100 SEK and their budget was also in SEK, the old system would convert 100 SEK to Mexican Pesos (200 MXN) and then back to SEK (99.98 SEK), losing precision in the process. The correct amount should have remained 100 SEK.

The Solution:

We changed the budget calculation logic to check if the transaction currency matches the budget currency before performing any conversion. If the currencies are the same, the system now uses the original transaction amount directly without any conversion. Currency conversion only happens when the currencies are different. This removes unnecessary conversions and prevents precision loss.

Bug 2: Large Amount Entry Without Warning (Found in Acceptance Testing)

The Problem:

Users could enter very large amounts when creating transactions or budgets without receiving any warning. According to the use case description, the system should warn users when they try to enter extremely large values to prevent data entry errors.

The Solution:

We added validation logic to detect very large amounts. When a user enters an amount that is too large, the system now shows a warning message asking the user to confirm if the amount is correct.

Effort Spent on Bug Removal

We spent a total of 3 hours on bug removal activities. Bug 1 took approximately 2 hours to identify, fix, and test. Bug 2 took approximately 1 hour.

Question 6: Confidence Level

We are highly confident that the product is ready for the demonstration. Our extensive testing of UC5 (Record Transaction) and UC8 (Create Budget) has made sure that these primary use cases are error-free and fully functional. Additionally, other features such as user management (UC1,

UC3, UC4), authentication (UC2), exchange rate management (UC13), and the feedback system (UC12, UC14) were also extensively tested with unit and integration tests.

All bugs identified during testing have been fixed and verified through our test suite. Our integration tests confirm that all components work together correctly, with 377 passing tests.

However, some of the features from the initial requirements specification could not be delivered in the final submission due to time constraints.